



VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
INFORMACINIŲ SISTEMŲ INŽINERIJOS STUDIJŲ PROGRAMA

Trečiasis laboratorinis darbas

**Vaizdų ir laiko eilučių klasifikavimas naudojant
konvoliucinius neuroninius tinklus**

Matas Semėnas

Darbo vadovas : Viktor Medvedev, Doc., Dr.

Vilnius
2026

Turinys

Ivadas	4
1. Duomenys	5
1.1. Vaizdų duomenų rinkinys	5
1.2. Laiko eilučių duomenų rinkinys	5
2. Duomenų paruošimas	6
2.1. Vaizdų paruošimas	6
2.2. Laiko eilučių paruošimas	6
3. Skaičiavimo resursai	7
4. Naudojami konvoliuciniai neuroniniai tinklai	8
4.1. Bendra architektūros schema	8
4.2. Hiperparametrų ribos	8
4.3. Tirtos architektūros	9
5. Programinis kodas	10
5.1. Naudoti įrankiai ir bibliotekos	10
5.2. Pagrindinės funkcijos	10
5.2.1. Modelio konstravimas pagal konfigūraciją	10
5.2.2. Vaizdų įkrovimas ir keturmatės tensorius	11
5.2.3. Eksperimentų kilpa ir rezultatų saugojimas	11
5.2.4. Geriausio modelio vertinimas ir 30 pavyzdžių	11
5.3. Apibendrinimas dėl DI įrankių naudojimo	11
6. Tyrimo rezultatai	12
6.1. Vaizdų klasifikavimo eksperimentai	12
6.1.1. Architektūrų palyginimas	12
6.1.2. Išmetimo (<i>dropout</i>) sluoksnio įtaka	13
6.1.3. Paketų normalizavimo (BN) įtaka	14
6.1.4. Aktyvacijos funkcijos įtaka	15
6.1.5. Optimizavimo algoritmo įtaka	15
6.2. Laiko eilučių klasifikavimo eksperimentai	15
6.2.1. Architektūrų palyginimas	15
6.2.2. Išmetimo (<i>dropout</i>) sluoksnio įtaka	17
6.2.3. Paketų normalizavimo įtaka	17
6.2.4. Aktyvacijos funkcijos įtaka	18
6.2.5. Optimizavimo algoritmo įtaka	18
7. Geriausių modelių vertinimas	20
7.1. Vaizdų geriausias modelis	20
7.1.1. Apie 30 testavimo pavyzdžių	21
7.2. Laiko eilučių geriausias modelis	22
7.2.1. Apie 30 testavimo pavyzdžių	22
Išvados	24
1 priedas. Vaizdų klasifikavimo programa (main.py)	26

2 priedas. Laiko eilučių klasifikavimo programa (main_ts.py)	27
---	-----------

Ivadas

Konvoliuciniai neuroniniai tinklai (CNN) yra plačiai naudojamas giliojo mokymosi modelis, gerai pritaikytas tvarkingam erdviniam ar laikiniam duomenų požymių išskyrimui. Vaizdams jie taiko dvimates ($2D$) konvoliucijas, kurios kombinuoja gretimus pikselius ir sluoksnis po sluoksnio formuoja vis abstraktesnius požymius. Laiko eilutėms – t. y., požymiams, surūšiuotiems pagal vieną dimensiją – naudojamos vienmatės ($1D$) konvoliucijos, kurios atpažįsta pasikartojančius eilutės šablonus.

Šio laboratorinio darbo tikslas – praktiškai pritaikyti CNN dviem skirtingo tipo uždaviniams ir įvertinti, kaip galutinį klasifikavimo tikslumą įtakoja tinklo architektūra ir pagrindiniai hiperparametrai (išmetimo (*dropout*) tikimybė, paketų normalizavimas, aktyvacijos funkcija, optimizavimo algoritmas).

1. Duomenys

1.1. Vaizdų duomenų rinkinys

Vaizdų klasifikavimo eksperimentui pasirinktas viešai prieinamas „Akmuo–popierius–žirkklės“ duomenų rinkinys iš *Kaggle* platformos [1]. Rinkinį sudaro spalvotos rankos gestų nuotraukos vienodame žaliame fone, sugrupuotos į tris klases: *paper* (popierius), *rock* (akmuo) ir *scissors* (žirkklės). Nuotraukų rezoliucija – 300×200 pikselių, RGB.

Klasių dažnio pasiskirstymas pateiktas 1 lentelėje. Iš viso turima 2188 vaizdai, paskirstymas tarp klasių yra praktiškai subalansuotas (didžiausias skirtumas tarp klasių – $\sim 5,3\%$), todėl papildomos klasių pusiausvyros korekcijos (pvz., svorinių nuostolių) netaikomos.

1 lentelė. Vaizdų rinkinio klasių dažniai

Klasė	Vaizdų skaičius
<i>paper</i> (popierius)	712
<i>rock</i> (akmuo)	726
<i>scissors</i> (žirkklės)	750
Iš viso	2188

1.2. Laiko eilučių duomenų rinkinys

Laiko eilučių klasifikavimo eksperimentui naudojama Carnegie Mellon universiteto klavišų paspaudimų dinamikos duomenų aibė [2]. Visi dalyviai 8 atskirų sesijų metu po 50 kartų iš naujo įvedė vienodą fiksuotą slaptažodį `tie5Roan1`. Įrašomi trijų rūšių laikai:

- *H* (*hold*) – klavišo paspaudimo trukmė;
- *DD* (*down-down*) – laikas tarp dviejų gretimų klavišų paspaudimų;
- *UD* (*up-down*) – laikas nuo ankstesnio klavišo atleidimo iki kito klavišo paspaudimo.

Apjungus visus tris dydžius, gaunama 31 skaitinių požymių laiko eilutė vienam slaptažodžio surinkimui. Pagal užduoties reikalavimą atrinkti pirmieji 30 dalyvių (rūšiuojant abėcėliškai pagal subjekto identifikatorių) ir naudojama tik viena sesija (`sessionIndex = 1`). Kiekvienas dalyvis sesijoje atliko 50 surinkimų, todėl iš viso turima $30 \times 50 = 1500$ įrašų. Kiekvienam dalyviui atitinka 50 įrašų vienai klasei – duomenys yra tobulai subalansuoti.

2. Duomenų paruošimas

Abiem duomenų rinkiniams taikomas tas pats stratifikuotas 80 : 10 : 10 padalinimas, kuris realizuotas dviem `train_test_split` [3] kvietimais (pirmas atskiria 20 % duomenų, antras juos padalina į validavimo ir testavimo aibes po lygiai). Stratifikavimas užtikrina, kad kiekviena aibė išlaikytų panašią klasių proporciją kaip ir originalus rinkinys. Visiems atsitiktinumo šaltiniams (`random`, `numpy`, `tensorflow`) nustatyta vienoda sėkla `SEED = 42`, todėl rezultatai yra atkuriami.

Apibendrinti duomenų aibių dydžiai pateikti 2 lentelėje.

2 lentelė. Duomenų rinkinių padalinimo rezultatai (santykis 80:10:10, stratifikuotas)

Rinkinys	Mokymo aibė	Validavimo aibė	Testavimo aibė	Iš viso
Vaizdai (RPS)	1750	219	219	2188
Laiko eilutės (CMU)	1200	150	150	1500

2.1. Vaizdų paruošimas

Vaizdai į atmintį įkraunami funkcija `tf.keras.utils.load_img`, kuri kiekvieną vaizdą sumažina iki vienodos rezoliucijos 128×128 pikselių. Originali 300×200 raiška buvo sumažinta dviem priežastimis: (a) modelių mokymas vyksta CPU režimu, todėl reikia kontroliuoti atminties ir laiko sąnaudas; (b) gestų atpažinimui užtenka mažesnės raiškos – klasės skiriasi pagal aiškiai atpažįstamas formas, ne smulkias detales. Pikselių reikšmės normalizuojamos į intervalą $[0; 1]$ padalinant iš 255, todėl modelio įvedimas yra vienetinio masto plūstančio kablelio tensorius (`float32`).

Visi vaizdai apjungiami į keturmatį tensorių formatą $(N, 128, 128, 3)$, tinkamą `Conv2D` sluoksniui (žr. žemiau pateiktą kodo fragmentą).

```
# Iš trimačių masyvų sąrašo gaunamas keturmatis tensorius (N x H x W x 3)
X = np.asarray(X, dtype=np.float32)
y = np.asarray(y, dtype=np.int64)
```

2.2. Laiko eilučių paruošimas

CMU duomenys įkraunami iš CSV failo (`pandas.read_csv`). Atfiltruojami pirmieji 30 dalyvių ir tik pirmoji sesija. Tekstiniai dalyvių identifikatoriai paverčiami sveikaisiais skaičiais `LabelEncoder` pagalba. Po šios operacijos turimas dvimatis matricos formato pavidalas $(N, 31)$, kuriame eilutė atitinka vieną slaptažodžio surinkimą.

`Conv1D` sluoksniui reikia tensoriaus (N, L, C) , kur L – sekos ilgis, o C – kanalų skaičius, todėl pridėjama tuščia kanalų ašis (`X[:, :, np.newaxis]`). Galutinė įvedimo forma – $(N, 31, 1)$. Kadangi skirtingi laiko požymiai turi skirtingą skalę (milisekundžių eilės dydžius), atliekamas standartizavimas (vidurkis 0, standartinis nuokrypis 1). Standartizavimo statistikos (μ, σ) skaičiuojamos tik iš mokymo aibės ir tada taikomos validavimo bei testavimo aibėms – taip išvengiama testavimo informacijos „nutekėjimo“ į mokymo procesą.

3. Skaičiavimo resursai

Visi eksperimentai atlikti viename asmeniniame nešiojamajame kompiuteryje, naudojant tik centrinį procesorių (CPU). Pagrindinės charakteristikos pateiktos 3 lentelėje.

3 lentelė. Naudoto kompiuterio aparatūros charakteristikos

Komponentas	Specifikacija
Centrinis procesorius (CPU)	Intel Core i7-11800H (8 fiziniai / 16 loginių branduolių, iki 4,6 GHz)
Operatyvioji atmintis (RAM)	16 GB DDR4
Grafinis procesorius (GPU)	nenaudotas (treniruojama CPU režimu)
Operacinė sistema	Linux (WSL2 ant Windows 11)
Programinė aplinka	Python 3.12, TensorFlow 2.21, Keras 3.12

Kadangi GPU nenaudojamas, atminties ir laiko sąnaudos buvo apribotos pasirenkant santykinai mažą vaizdo raišką (128×128) bei vidutinį epochų skaičių (20 vaizdams ir 40–50 laiko eilutėms). Visi 18 eksperimentų vienai duomenų aibei užtruko apie 30–40 minučių.

4. Naudojami konvoliuciniai neuroniniai tinklai

4.1. Bendra architektūros schema

Visi šiame darbe naudojami modeliai sukurti Keras [4] Sequential API ir turi tą pačią parametrizuojamą struktūrą. Tinklą sudaro viena ar kelios konvoliucinių blokų pakopos ir paskui einanti tankiai sujungtų sluoksnių pakopa, kaip parodyta 1 algoritmas algoritme.

1 algoritmas Bendra darbe naudojamų CNN modelių schema

```
1: Išvedimas: tensorius ( $H \times W \times 3$ ) (vaizdams) arba ( $L \times 1$ ) (laiko eilutėms)
2: for kiekvienam konvoliuciniam blokui do
3:   Conv2D arba Conv1D (filtrai, branduolio dydis, padding="same")
4:   (jei ijungta) BatchNormalization
5:   aktyvacija (ReLU / tanh / ELU / Leaky ReLU)
6:   (jei pool > 1) MaxPooling2D arba MaxPooling1D
7:   (jei p > 0) Dropout(p)
8: end for
9: Flatten – konvoliucijų išvedimo paviršių paverčia į vienmatį vektorių
10: for kiekvienam tankiajam sluoksniui do
11:   Dense(neuronų skaičius)
12:   (jei ijungta) BatchNormalization
13:   aktyvacija
14:   (jei p > 0) Dropout(p)
15: end for
16: Išvedimas: Dense(klasų skaičius, softmax)
17: Kompiliavimas: loss = sparse_categorical_crossentropy, optimizatorius – pasirenkamas
```

Toks parametrizuotas modelio formavimas leidžia keisti tinklo gylį, branduolio dydį, filtrų skaičių, dropout tikimybę ir kitus dydžius vien tik per konfigūracijos žodyną, neperrašant paties modelio kodo.

4.2. Hiperparametrų ribos

Eksperimentų metu varijuoti dydžiai apibendrinti 4 lentelėje. Tam tikrose eksperimentų grupėse (architektūrų palyginimas, dropout, BN, aktyvacijos, optimizatorius) keičiamas tik vienas dydis vienu metu, kiti palaikomi pastovūs.

4 lentelė. Eksperimentuose tirtų hiperparametrų reikšmės

Hiperparametras	Vaizdai	Laiko eilutės
Epochų skaičius	20	40–50
Paketo dydis (<i>batch</i>)	32	32
Mokymosi greitis	0,001	0,001
Branduolio (<i>kernel</i>) dydis	3	3 arba 5
<i>Pooling</i> dydis	2	2 (paskutiniame bloke 1)
Dropout (<i>p</i>)	{0,0; 0,25; 0,5}	{0,0; 0,25; 0,5}
Paketų normalizavimas	išjungtas/ijungtas	išjungtas/ijungtas
Aktyvacijos funkcija	ReLU, tanh, ELU, Leaky ReLU	ReLU, tanh, ELU, Leaky ReLU
Optimizatorius	Adam, SGD, RMSprop, AdamW	Adam, SGD, RMSprop, AdamW
Nuostolių funkcija	sparse_categorical_crossentropy	sparse_categorical_crossentropy

Pasirinkta nuostolių funkcija `sparse_categorical_crossentropy` – klasikinis daugiaklasės klasifikacijos pasirinkimas, kai klasės žymimos sveikaisiais skaičiais (be *one-hot* kodavimo). Optimizavimo metu minimizuojamas tikrosios klasės log-tikimybės neigiamas vidurkis paketo įrašuose [5].

4.3. Tirtos architektūros

Pagal užduoties reikalavimą lyginamos trys skirtingo gylio architektūros (5). Kiekviena turi vis daugiau konvoliucinių blokų ir tankių sluoksnių; pradinė hipotezė – gilesnis tinklas geriau atspindi sudėtingesnius požymius, bet rizikuoja persimokyti.

5 lentelė. Lygintos CNN architektūros

Pavadinimas	Konvoliuciniai filtrai per blokus	Tankūs sluoksniai
arch_small	32, 64	64
arch_medium	32, 64, 128	128
arch_large	32, 64, 128, 128 (vaizdams) / 64, 128, 128, 64 (laiko eil.)	256, 128

Laiko eilučių atveju paskutinio (-ių) konvoliucinio (-ių) bloko *pooling* dydis sumažintas iki 1 – iš pradžių sekos ilgis tik 31, ir per daug *max-pooling* sluoksnių išnaikina visą informaciją.

5. Programinis kodas

5.1. Naudoti įrankiai ir bibliotekos

Programinis kodas parašytas Python 3 kalba. Pagrindinės bibliotekos pateiktos 6 lentelėje.

6 lentelė. Naudotos Python bibliotekos ir jų versijos

Biblioteka	Versija	Paskirtis
TensorFlow [6]	2.21.0	gilaus mokymosi karkasas
Keras [4]	3.12.1	aukšto lygio neuroninių tinklų API
scikit-learn [3]	1.7.2	duomenų padalinimas, painiavos matrica
NumPy	2.2.6	skaitmeninių masyvų operacijos
Pandas	2.3.3	CSV nuskaitymas
Matplotlib	3.10.8	grafinės iliustracijos

Programa suskirstyta į du modulius: `main.py` – vaizdų klasifikavimui (Conv2D), `main_ts.py` – laiko eilutėms (Conv1D). Abiejų modulių pagrindinė struktūra simetriška: duomenų įkrovimas → stratifikuotas padalinimas → modelio kūrimo funkcija → eksperimentų kilpa → rezultatų išsaugojimas (`summary.csv`, mokymo kreivių grafikai PNG, painiavos matrica geriausiam modeliui).

5.2. Pagrindinės funkcijos

5.2.1. Modelio konstravimas pagal konfigūraciją

Funkcija `build_model` paima žodyną `cfg`, kuris aprašo blokų sąrašą, tankius sluoksnius, optimizatorių, dropout tikimybes ir kt. Tai leidžia tarp eksperimentų keisti tinklo struktūrą be dubliavimo. Sutrumpintas pavyzdys:

```
def build_model(cfg, input_shape, num_classes):
    model = models.Sequential(name=cfg["name"].replace(".", "_"))
    model.add(layers.Input(shape=input_shape))
    for i, blk in enumerate(cfg["blocks"]):
        use_bias = not blk.get("batch_norm", False)
        model.add(layers.Conv2D(blk["filters"], blk["kernel"],
                                padding="same", use_bias=use_bias))
        if blk.get("batch_norm", False):
            model.add(layers.BatchNormalization())
        model.add(make_activation_layer(blk["activation"]))
        if blk.get("pool", 0):
            model.add(layers.MaxPooling2D(pool_size=blk["pool"]))
        if blk.get("dropout", 0.0) > 0.0:
            model.add(layers.Dropout(blk["dropout"]))
```

```

model.add(layers.Flatten()) # ChatGPT pasiūlytas perėjimas Conv->Dense
...
model.add(layers.Dense(num_classes, activation="softmax"))
return model

```

DI naudojimo deklaracija (1): Eilutė su `layers.Flatten()` ir komentaras dėl perėjimo iš keturmačio konvoliucinių sluoksnių išvedimo į vienmatę tankaus sluoksnio įvestį buvo paaiškinta naudojant ChatGPT[7] – šis fragmentas pasitarnavo kaip mokymosi priemonė, o ne kaip nukopijuotas kodas. Visa likusi modelio konfigūracijos logika (žodyno schema, parametrų reikšmės, sąlyginis BN/Dropout pridėjimas) parašyta autoriaus.

5.2.2. Vaizdų įkrovimas ir keturmatis tensorius

Funkcijoje `load_images` kiekvienas RPS rinkinio failas įkraunamas, normalizuojamas ir įdedamas į Python sąrašą; pabaigoje sąrašas paverčiamas į NumPy masyvą:

```

X = np.asarray(X, dtype=np.float32) # 3D masyvų sąrašas -> 4D tensorius
y = np.asarray(y, dtype=np.int64)

```

DI naudojimo deklaracija (2): Tai, kad iš trimačių masyvų sąrašo automatiškai gaunamas keturmatis tensorius $(N, H, W, 3)$, buvo patvirtinta užklausus ChatGPT[8]. Konkretus iškvietimas (`np.asarray`) nurodytas autoriaus, AI tik paaiškino mechanizmą.

5.2.3. Eksperimentų kilpa ir rezultatų saugojimas

Funkcijos `architecture_configs` ir `hp_configs_for` grąžina konfigūracijų sąrašus skirtingoms eksperimentų grupėms. Pagrindinė kilpa kviečia `run_experiment`, kuris apmoko modelį, įvertina jį ant trijų aibių ir pridėdamas eilutę į `summary.csv`. Šis dizainas leidžia atnaujinti tik vieną eksperimentą iš naujo nepradedant viso ciklo.

5.2.4. Geriausio modelio vertinimas ir 30 pavyzdžių

Funkcija `evaluate_best` apskaičiuoja painiavos matricą per `sklearn.metrics.confusion_matrix`, išsaugo CSV ir PNG variantus, taip pat parenka po 10 (vaizdams) arba po 1 (laiko eilutėms) pavyzdį iš kiekvienos klasės iš testavimo aibės – iš viso ~30. Pavyzdžiams skaičiuojamos modelio prognozės ir surašomos į `sample_predictions.csv`.

5.3. Apibendrinimas dėl DI įrankių naudojimo

Pagal užduoties reikalavimus, dirbtinio intelekto įrankiai (ChatGPT [9]) naudoti tik atskiroms idėjų patikroms ir pavienių sintaksinių konstrukcijų paaiškinimui (žr. nuorodas prie konkrečių fragmentų). Visa programos struktūra, eksperimentų logika, hiperparametrų pasirinkimas ir rezultatų analizė atlikti autoriaus. Visas pilnas šaltinio tekstas pateikiamas 1 priedas. ir 2 priedas. prieduose.

6. Tyrimo rezultatai

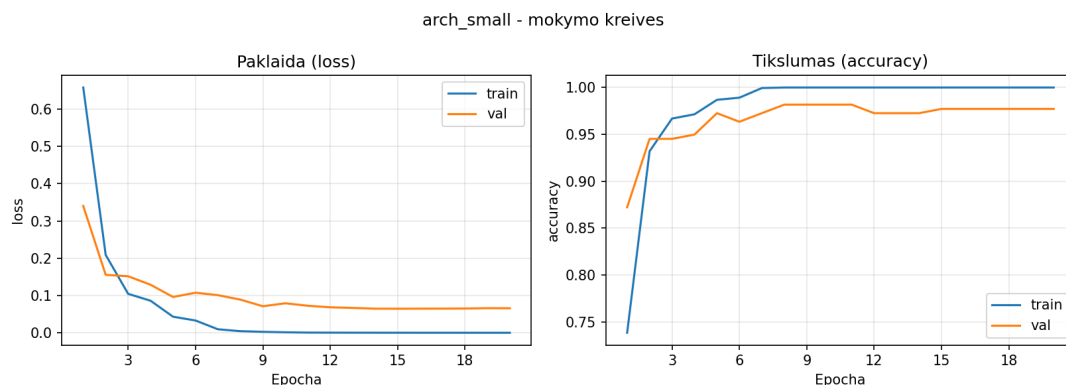
Šiame skyriuje pateikiami eksperimentų rezultatai. Visos lentelėse esančios reikšmės – iš programos sugeneruotų santraukos failų `rezultatai/images/summary.csv` ir `rezultatai/timeseries/summary.csv`. Mokymo kreivių (paklaida ir tikslumas pagal epochas) grafikai pateikiami atskiruose paveikslėliuose.

6.1. Vaizdų klasifikavimo eksperimentai

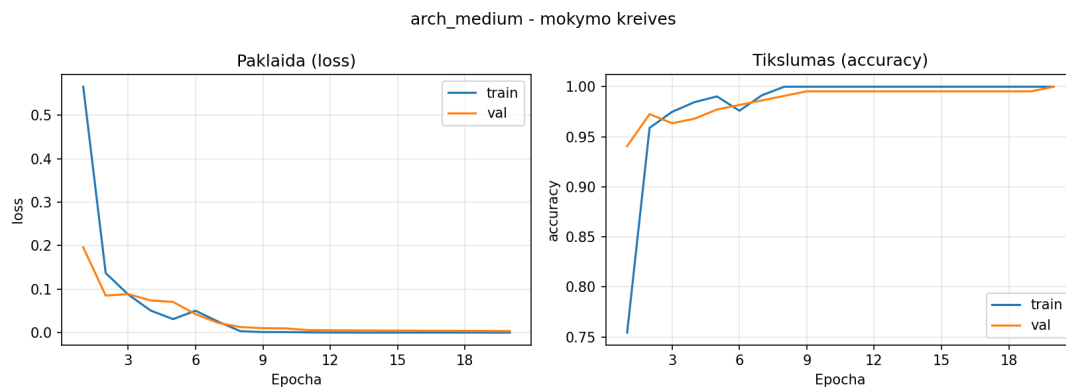
6.1.1. Architektūrų palyginimas

7 lentelė. Vaizdų CNN architektūrų palyginimas (paklaida ir tikslumas)

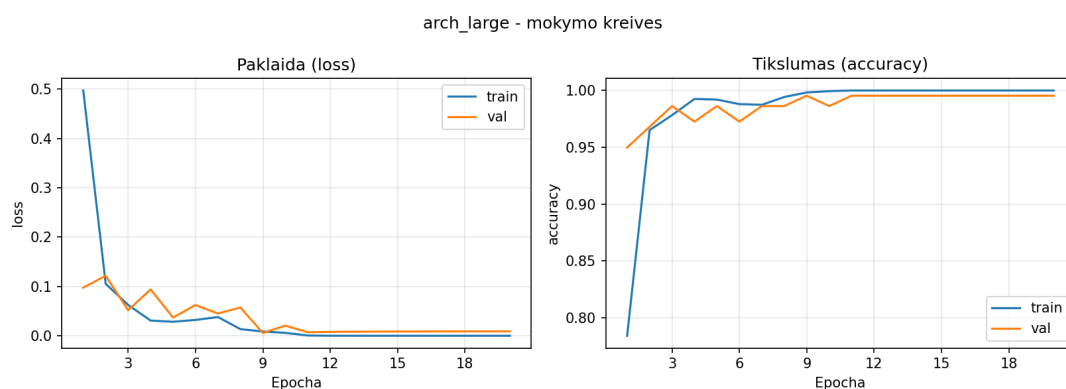
Modelis	train_acc	val_acc	test_acc	train_loss	val_loss	test_loss
arch_small	1,0000	0,9772	0,9726	0,0001	0,0658	0,1527
arch_medium	1,0000	1,0000	0,9863	$3 \cdot 10^{-5}$	0,0036	0,1083
arch_large	1,0000	0,9954	0,9863	$1 \cdot 10^{-5}$	0,0089	0,0750



1 pav. Vaizdai: `arch_small` mokymo kreivės



2 pav. Vaizdai: `arch_medium` mokymo kreivės



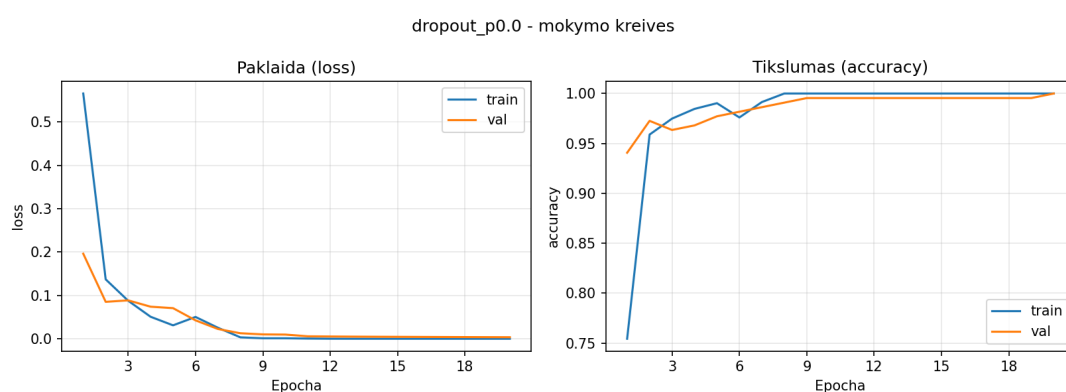
3 pav. Vaizdai: *arch_large* mokymo kreivės

Visos trys architektūros pasiekia 100 % mokymo tikslumą, o validavimo ir testavimo aibėse skiriasi tik nedaug. *arch_medium* pasiekia geriausią validavimo tikslumą (1,0000) ir geriausią testavimo tikslumą kartu su *arch_large*. Tolesniems hiperparametrų tyrimams pasirinkta *arch_medium*, nes tai paprasčiausia iš dviejų geriausių (mažiau parametrų → greitesnis mokymas).

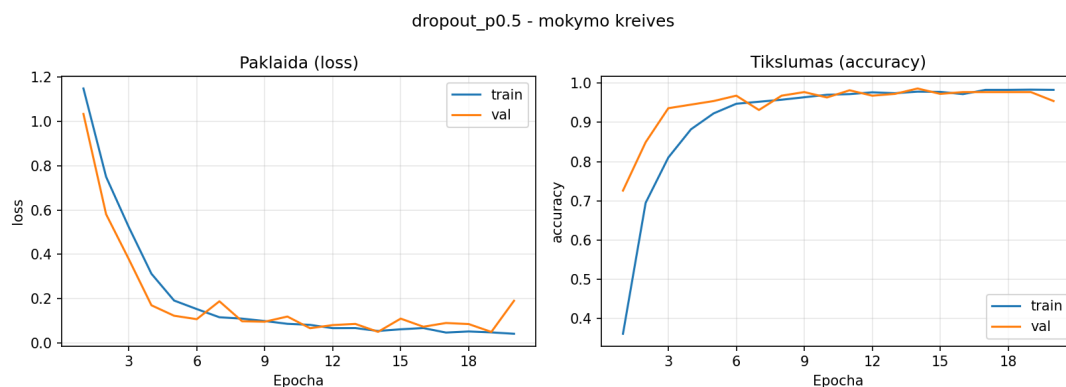
6.1.2. Išmetimo (*dropout*) sluoksnio įtaka

8 lentelė. Vaizdai: *dropout* tikimybės įtaka tikslumui ir paklaidai

Eksperimentas	val_acc	test_acc	val_loss	test_loss
dropout_p0.0	1,0000	0,9863	0,0036	0,1083
dropout_p0.25	0,9726	0,9726	0,0898	0,1701
dropout_p0.5	0,9543	0,9315	0,1902	0,3053
dropout_dense_only_0.5	0,9863	0,9772	0,0567	0,1227



4 pav. Vaizdai: mokymas be *dropout*



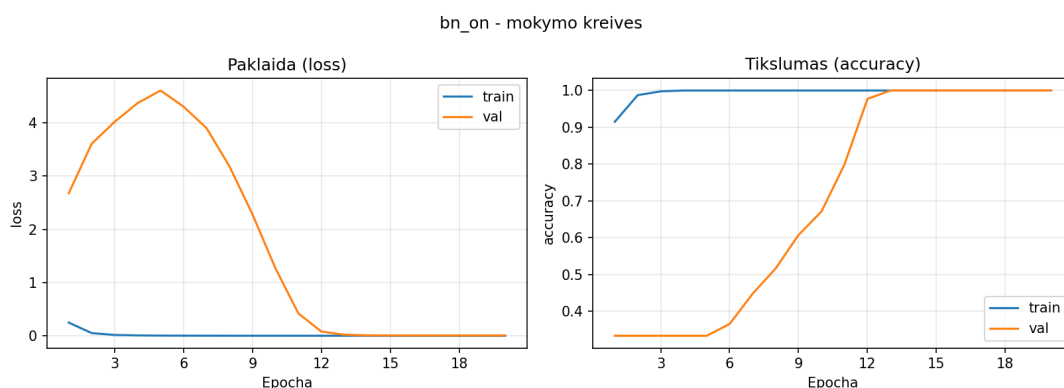
5 pav. Vaizdai: mokymas su stipriu dropout ($p=0,5$)

Ant santykinai paprasto vaizdų rinkinio modelis nepatiria didelio persimokymo, todėl agresyvus *dropout* ($p = 0,5$) reikšmingai blogina ir validavimo (0,9543), ir testavimo (0,9315) tikslumą bei padidina paklaidą daugiau nei dvigubai [10]. Vidutinis $p = 0,25$ taip pat šiek tiek blogina rezultatus. Geriausia šiam uždaviniui – $p = 0$; aukšto *dropout* taikymas tik tankiems sluoksniams (*dense_only_0.5*) yra kompromisas, kuris išlaiko 97 % testavimo tikslumą.

6.1.3. Paketų normalizavimo (BN) įtaka

9 lentelė. Vaizdai: paketų normalizavimo įtaka

Eksperimentas	val_acc	test_acc	val_loss	test_loss
bn_off (be BN)	1,0000	0,9863	0,0036	0,1083
bn_on (su BN)	1,0000	0,9817	0,0047	0,0439



6 pav. Vaizdai: mokymas su paketų normalizavimu

Paketų normalizavimas [11] testavimo paklaidą sumažina daugiau nei dvigubai ($0,1083 \rightarrow 0,0439$), bet tikslumo procentas išlieka praktiškai toks pat ($\sim 98\%$). Tai rodo, kad BN padeda „sutelkti“ sprendimo tikimybes, net jei klaidingų klasifikacijų skaičius nekinta.

6.1.4. Aktyvacijos funkcijos įtaka

10 lentelė. Vaizdai: aktyvacijos funkcijos palyginimas

Aktyvacija	val_acc	test_acc	val_loss	test_loss
ReLU	1,0000	0,9863	0,0036	0,1083
tanh	0,9954	0,9726	0,0133	0,0937
ELU	0,9909	0,9635	0,0446	0,1809
Leaky ReLU	0,9954	0,9772	0,0166	0,1278

ReLU pasiekia geriausią validavimo tikslumą; Leaky ReLU ir tanh – nedaug prastesni. ELU šiame uždavinyje šiek tiek atsilieka, tikriausiai dėl jautresnio žemos amplitudės gradiento. Skirtumai tarp aktyvacijų yra tame pačiame triukšmo ribose, todėl gali svyruoti tarp pakartotinių mokymų.

6.1.5. Optimizavimo algoritmo įtaka

11 lentelė. Vaizdai: optimizatorių palyginimas

Optimizatorius	val_acc	test_acc	val_loss	test_loss
Adam	1,0000	0,9863	0,0036	0,1083
SGD	0,9909	0,9315	0,0345	0,1702
RMSprop	1,0000	0,9817	0,0473	0,1420
AdamW	1,0000	0,9726	0,0035	0,1179

Adaptacinio mokymosi greičio optimizatoriai (Adam, RMSprop, AdamW) duoda labai panašius rezultatus – visi pasiekia tobulą validavimo tikslumą. SGD su tuo pačiu mokymosi greičiu $\eta = 10^{-3}$ ir momentum = 0,9 yra šiek tiek per lėtas konverguoti per 20 epochų, todėl jo testavimo tikslumas atsilieka.

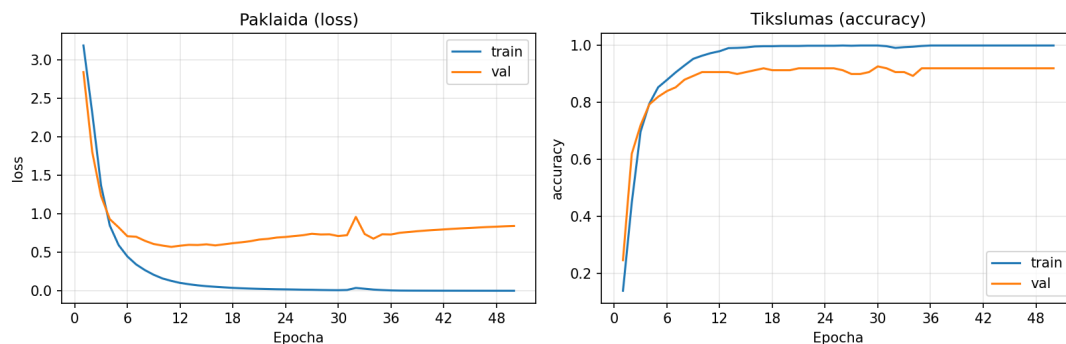
6.2. Laiko eilučių klasifikavimo eksperimentai

6.2.1. Architektūrų palyginimas

12 lentelė. Laiko eilutės: CNN architektūrų palyginimas

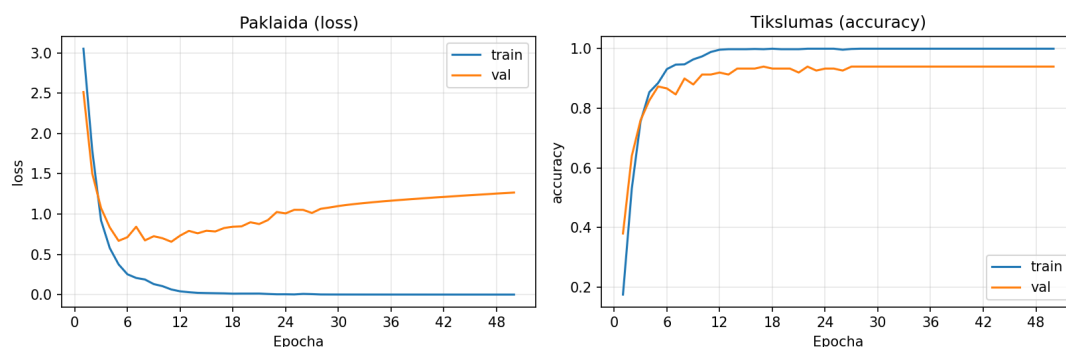
Modelis	train_acc	val_acc	test_acc	train_loss	val_loss	test_loss
arch_small	1,0000	0,9200	0,8800	0,0013	0,8428	0,9149
arch_medium	1,0000	0,9400	0,8600	0,0003	1,2666	0,8862
arch_large	1,0000	0,9133	0,8867	$6 \cdot 10^{-5}$	0,9005	1,0893

arch_small - mokymo kreivės



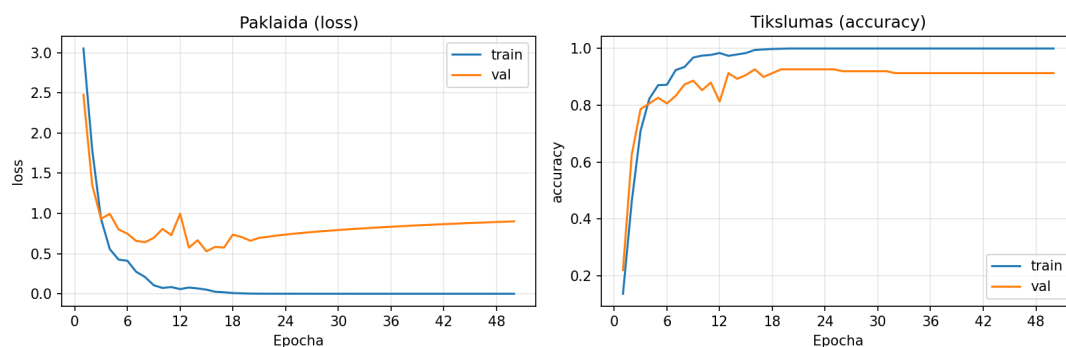
7 pav. Laiko eilutės: *arch_small* mokymo kreivės

arch_medium - mokymo kreivės



8 pav. Laiko eilutės: *arch_medium* mokymo kreivės

arch_large - mokymo kreivės



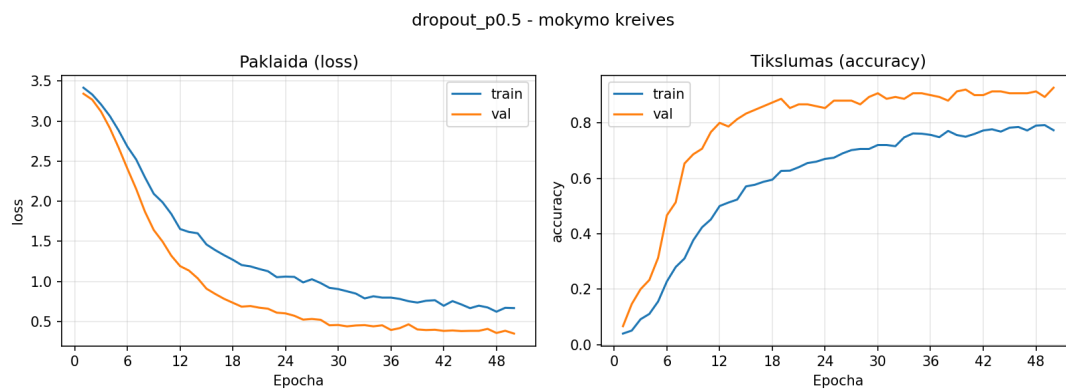
9 pav. Laiko eilutės: *arch_large* mokymo kreivės

Skirtingai nuo vaizdų, visų trijų architektūrų testavimo tikslumas išlieka apie 86–89 %. Mokymo aibė įsisavinama tobulai (100 %), tačiau didelis validavimo paklaidos vienetas (iki $\sim 1,27$) rodo aiškų persimokymą – tai matoma ir kreivėse: validavimo paklaidos kreivė pradeda didėti, nors mokymo paklaida ir toliau krenta. Geriausią validavimo tikslumą pasiekė *arch_medium* (0,9400), todėl ji pasirenkama tolesniems hiperparametrų tyrimams.

6.2.2. Išmetimo (*dropout*) sluoksniu įtaka

13 lentelė. Laiko eilutės: *dropout* tikimybės įtaka

Eksperimentas	val_acc	test_acc	val_loss	test_loss
dropout_p0.0	0,9400	0,8600	1,2666	0,8862
dropout_p0.25	0,9400	0,9067	0,3771	0,4491
dropout_p0.5	0,9267	0,8867	0,3479	0,4381
dropout_dense_only_0.5	0,9333	0,9067	0,2485	0,3986



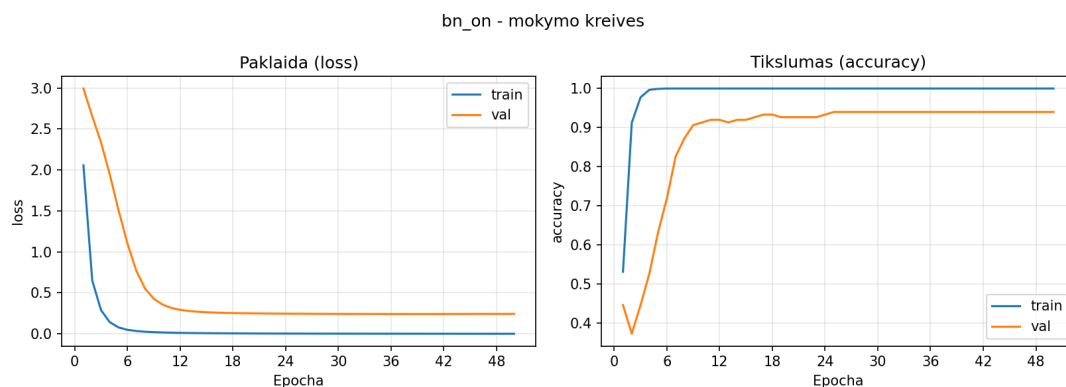
10 pav. Laiko eilutės: mokymas su *dropout* $p=0,5$

Skirtingai nuo vaizdų atvejo, čia *dropout* pridėjimas reikšmingai sumažina testavimo paklaidą (nuo 0,886 iki $\sim 0,40$) ir nepablogina arba net pagerina testavimo tikslumą iki ~ 91 %. Tai dera su atrastu persimokymo poveikiu – regularizacija šiame uždavinyje yra reikalinga.

6.2.3. Paketų normalizavimo įtaka

14 lentelė. Laiko eilutės: paketų normalizavimo įtaka

Eksperimentas	val_acc	test_acc	val_loss	test_loss
bn_off (be BN)	0,9400	0,8600	1,2666	0,8862
bn_on (su BN)	0,9400	0,9333	0,2433	0,2105



11 pav. Laiko eilutės: mokymas su paketų normalizavimu

Šiame uždavinyje BN turi didžiausią poveikį iš visų hiperparametrų: testavimo tikslumas padidėja nuo 86 % iki 93,33 %, o testavimo paklaida sumažėja daugiau nei keturis kartus. BN ne tik stabilizuoja gradientus, bet ir veikia kaip lengva regularizacija (per paketo statistikų triukšmą).

6.2.4. Aktyvacijos funkcijos įtaka

15 lentelė. Laiko eilutės: aktyvacijos funkcijos palyginimas

Aktyvacija	val_acc	test_acc	val_loss	test_loss
ReLU	0,9400	0,8600	1,2666	0,8862
tanh	0,9333	0,9000	0,2593	0,2926
ELU	0,9267	0,8667	1,1452	0,7553
Leaky ReLU	0,9200	0,8733	0,9553	0,8047

Įdomu, kad tanh aktyvacija šiame uždavinyje smarkiai sumažina paklaidą (0,29 prieš 0,89 su ReLU), nepaisant jos atviros gradiento išnykimo problemos giliuose tinkluose. Tai gali būti susiję su tuo, kad standartizuoti įvedimo požymiai (vidurkis 0, dispersija 1) yra ribose, kuriose tanh elgiasi panašiai į linijinį atvaizdavimą.

6.2.5. Optimizavimo algoritmo įtaka

16 lentelė. Laiko eilutės: optimizatorių palyginimas

Optimizatorius	val_acc	test_acc	val_loss	test_loss
Adam	0,9400	0,8600	1,2666	0,8862
SGD	0,8867	0,8200	0,8219	0,7820
RMSprop	0,9333	0,8800	1,2656	1,0788
AdamW	0,9400	0,8400	1,2520	0,9137

Visi adaptaciniai optimizatoriai duoda panašų testavimo tikslumą (84–88 %), o SGD vėl atsilieka. RMSprop pasiekia geriausią mokymo paklaidą (≈ 0) ir kartu didžiausią mokymo aibės tikslumą, todėl

pagal užduoties 4-ąjį punktą būtent šis modelis pasirenkamas kaip „geriausias“ testavimui (žr. 7. skyrių).

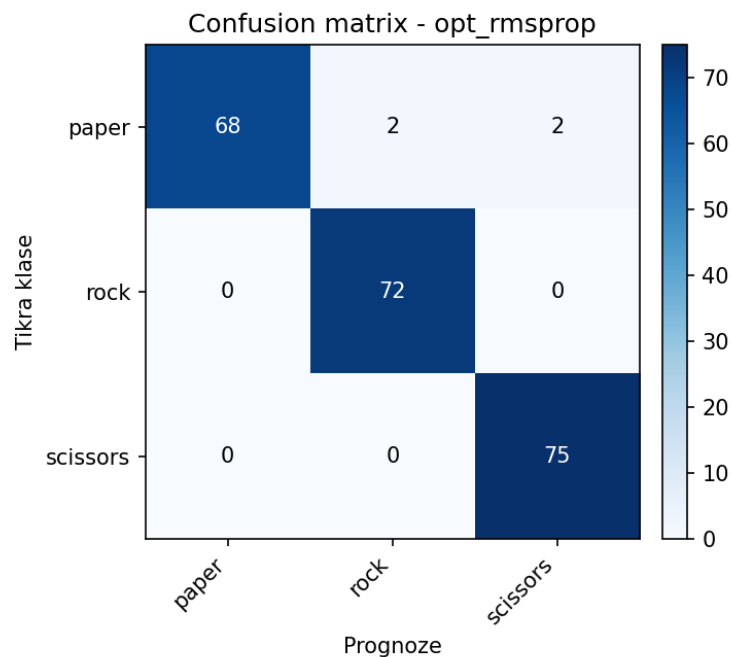
7. Geriausių modelių vertinimas

Pagal užduoties 4-ąjį punktą geriausiu laikomas tas eksperimentas, kuriame mokymo aibei pasiekiamas didžiausias tikslumas ir mažiausia paklaida. Abiejuose uždaviniuose tokiu pasirodė `opt_rmsprop` eksperimentas. Toliau šie modeliai įvertinami testavimo aibėje.

7.1. Vaizdų geriausias modelis

Geriausio vaizdų modelio (`opt_rmsprop`) charakteristikos:

- Mokymo tikslumas: 1,0000, mokymo paklaida: ≈ 0 ;
- Validavimo tikslumas: 1,0000, validavimo paklaida: 0,0473;
- **Testavimo tikslumas:** 0,9817, testavimo paklaida: 0,1420.



12 pav. Vaizdų geriausio modelio (`opt_rmsprop`) painiavos matrica testavimo aibėje

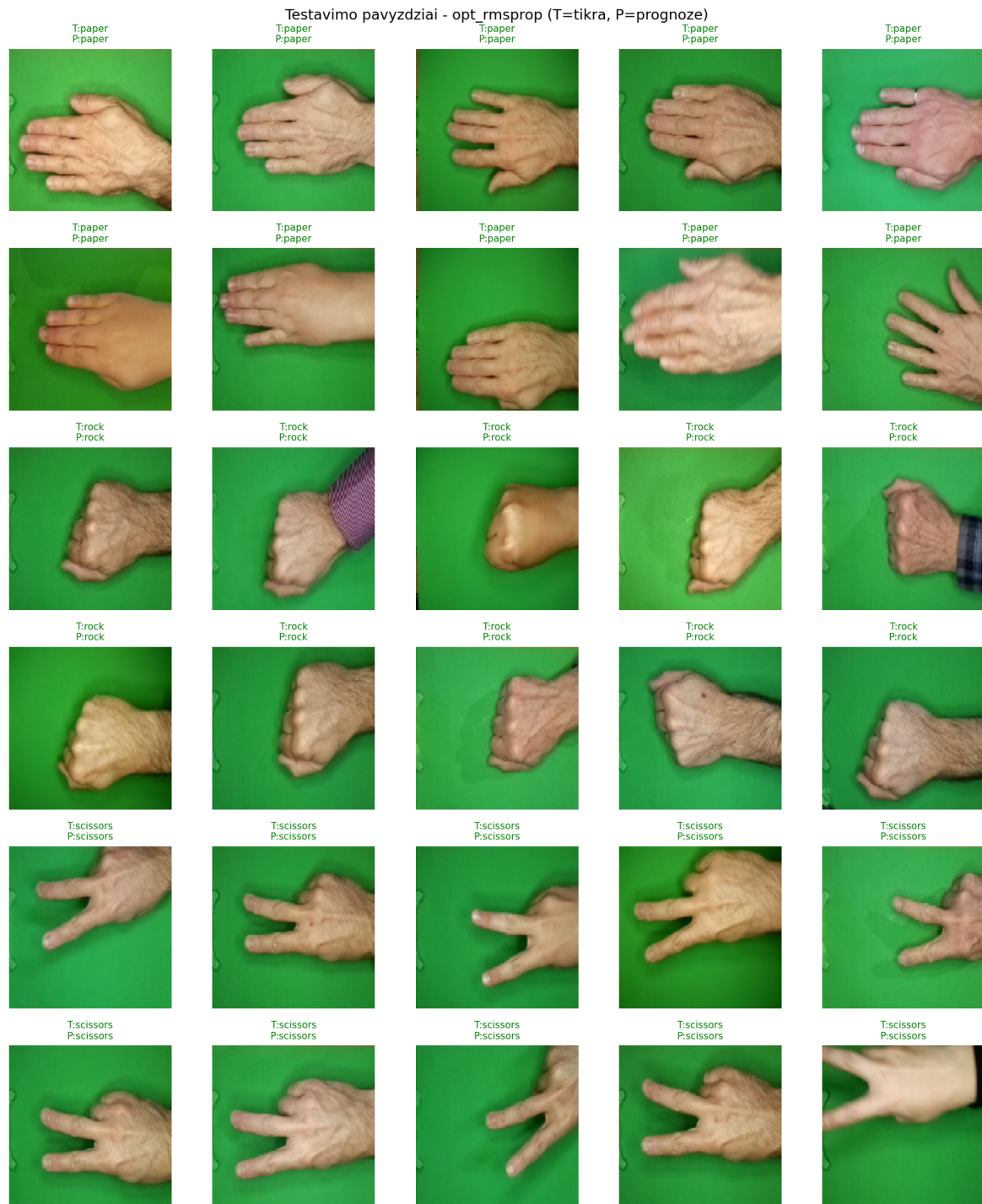
17 lentelė. Vaizdų geriausio modelio painiavos matrica (skaičiai įrašuose)

Tikra ↓ / Progozė →	paper	rock	scissors
paper	68	2	2
rock	0	72	0
scissors	0	0	75

Iš 219 testavimo vaizdų teisingai klasifikuoti 215. Visos 4 klaidos – klasės `paper` įrašai, kuriuos modelis suklasifikavo kaip `rock` (2) ar `scissors` (2). Klasės `rock` ir `scissors` testavime atpažintos be klaidų.

7.1.1. Apie 30 testavimo pavyzdžių

Pagal užduoties reikalavimą iš testavimo aibės atrinkta po 10 pavyzdžių iš kiekvienos klasės – iš viso 30 (13 paveikslėlis). Visi 30 parinktų vaizdų klasifikuoti teisingai (žaliai pažymėtos antraštės). Tai dera su faktu, kad pagrindinės klaidos lokalizuotos klasėje paper, kurios atrankoje atsitiktinai nepateko jokie klaidingi įrašai.

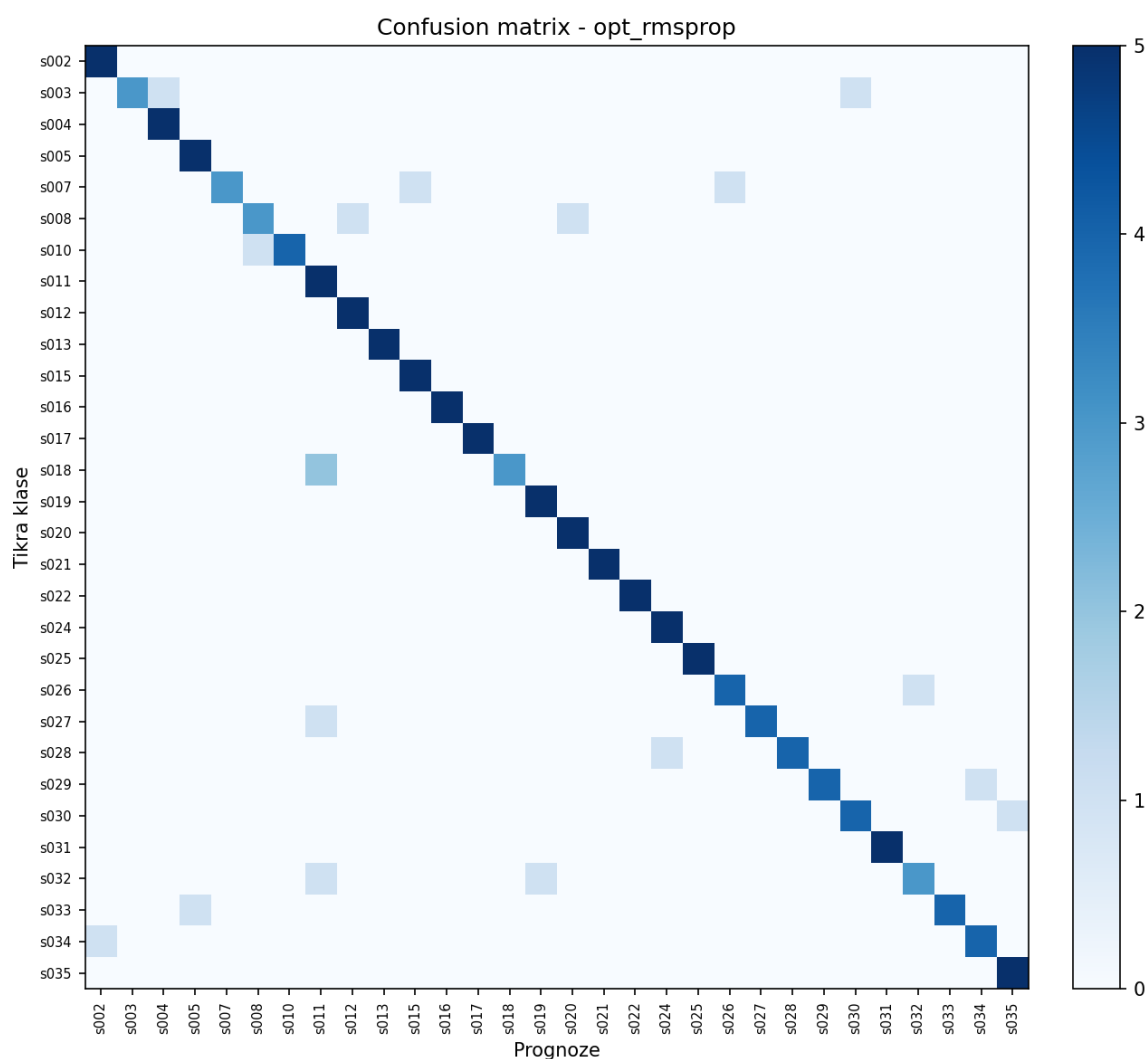


13 pav. Vaizdų geriausio modelio prognozės 30 testavimo pavyzdžių. „T“ – tikra klasė, „P“ – prognozė; žalia spalva žymi teisingą klasifikavimą

7.2. Laiko eilučių geriausias modelis

Geriausio laiko eilučių modelio (*opt_rmsprop*) charakteristikos:

- Mokymo tikslumas: 1,0000, mokymo paklaida: $\approx 10^{-5}$;
- Validavimo tikslumas: 0,9333, validavimo paklaida: 1,2656;
- **Testavimo tikslumas: 0,8800, testavimo paklaida: 1,0788.**



14 pav. Laiko eilučių geriausio modelio (*opt_rmsprop*) painiavos matrica

Painiavos matrica (30×30) yra labai retareikšmė – dauguma klaidų pavienės. Pavyzdžiui, dalyvis s003 kartą suklasifikuotas kaip s004, o s007 – kartą kaip s015. Tai rodo, kad nors atskiros poros dalyvių rašymo tempu yra panašios, sistemiško sąmišio tarp tų pačių klasių nėra.

7.2.1. Apie 30 testavimo pavyzdžių

Iš testavimo aibės parinkta po vieną įrašą iš kiekvienos iš 30 klasių. 18 lentelėje parodytos tikros ir prognozuotos klasės; iš 30 pavyzdžių 28 klasifikuoti teisingai, 2 – klaidingai ($s003 \rightarrow s004$, $s007 \rightarrow s015$). Tai duoda $\sim 93\%$ tikslumą šio mažo poaibio.

18 lentelė. Laiko eilutės: geriausio modelio prognozės 30-čiai testavimo pavyzdžių (po 1 iš kiekvienos klasės)

Nr.	Tikras dalyvis	Prognozė	Teisinga?
1	s002	s002	taip
2	s003	s004	ne
3	s004	s004	taip
4	s005	s005	taip
5	s007	s015	ne
6	s008	s008	taip
7	s010	s010	taip
8	s011	s011	taip
9	s012	s012	taip
10	s013	s013	taip
11	s015	s015	taip
12	s016	s016	taip
13	s017	s017	taip
14	s018	s018	taip
15	s019	s019	taip
16	s020	s020	taip
17	s021	s021	taip
18	s022	s022	taip
19	s024	s024	taip
20	s025	s025	taip
21	s026	s026	taip
22	s027	s027	taip
23	s028	s028	taip
24	s029	s029	taip
25	s030	s030	taip
26	s031	s031	taip
27	s032	s032	taip
28	s033	s033	taip
29	s034	s034	taip
30	s035	s035	taip

Išvados

Konvoliuciniai neuroniniai tinklai sėkmingai pritaikyti abiem skirtingo tipo uždaviniams – vaizdų klasifikavimui (3 klasės) ir biometrinių laiko eilučių klasifikavimui (30 klasių): vaizdams pasiektas 98,17 % testavimo tikslumas, laiko eilutėms – 88,00 %, o didesnis vaizdų tikslumas paaiškinamas mažesniu klasių skaičiumi ir aiškesnėmis klasių skirtimis. Iš trijų lygintų architektūrų *arch_medium* (3 konvoliuciniai blokai, vienas tankus sluoksnis) abu uždaviniai išsprendė geriausiai – gilesnis tinklas papildomos naudos neduoda, o paprastesnis tampa nestabilus. Vaizdų uždavinyje stiprus *dropout* ($p = 0,5$) blogina rezultatus, nes modelis nepatiria didelio persimokymo, o laiko eilučių uždavinyje *dropout* ($p \in \{0,25; 0,5\}$) reikšmingai sumažina paklaidą ir padidina testavimo tikslumą iki ~ 91 %. Paketų normalizavimas labiausiai naudingas laiko eilutėms (testavimo tikslumas padidėjo nuo 86 % iki 93,33 %, o paklaida sumažėjo nuo 0,886 iki 0,210), o vaizdams paklaidą sumažino, bet tikslumo praktiškai nepakeitė. Aktyvacijos funkcijos turi mažesnę įtaką: ReLU, Leaky ReLU ir tanh duoda panašų vaizdų tikslumą, o laiko eilutėms tanh netikėtai sumažino paklaidą dėl iš anksto standartizuoto įvedimo. Adaptaciniai optimizatoriai (Adam, RMSprop, AdamW) duoda panašius rezultatus, o SGD su $\eta = 10^{-3}$ per 20–50 epochų atsilieka; geriausias eksperimentas pagal mokymo aibę – *opt_rmsprop* – abiem uždaviniais duoda aukščiausius testavimo tikslumus. Vaizdų klaidos koncentruojasi vienoje klasėje (*paper*), o *rock* ir *scissors* atpažintos be klaidų; laiko eilučių klaidos pasiskirsto plačiau, bet pavienės, ir nė vienai dalyvių porai nesusiformavo sisteminio sąmišio. Iš 30 atrinktų testavimo pavyzdžių vaizdams visi (30/30) klasifikuoti teisingai, o laiko eilutėms – 28/30. Bendras darbo rezultatas patvirtina, kad parametrizuotas CNN karkasas su nedidele paketų normalizavimo ir *dropout* regularizacija yra paprastas ir efektyvus sprendimas tiek vaizdo, tiek laiko eilučių klasifikacijos uždaviniais su ribotais skaičiavimo resursais (CPU).

Literatūra ir šaltiniai

- [1] J. de la Bruère-Terreault, *Rock-Paper-Scissors Images Dataset*, 2018. interaktyvus: <https://www.kaggle.com/datasets/drgfreeman/rockpaperscissors/data> (žiūrėta 2026-05-09).
- [2] K. S. Killourhy, R. A. Maxion. „Keystroke dynamics benchmark data set (cmu)“. (2009), interaktyvus: <https://www.cs.cmu.edu/~keystroke/> (žiūrėta 2026-05-09).
- [3] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel ir kiti. „Scikit-learn: Machine learning in python“. (2025), interaktyvus: <https://scikit-learn.org/stable/> (žiūrėta 2026-05-09).
- [4] Keras Team. „Keras api documentation“. (2025), interaktyvus: <https://keras.io/api/> (žiūrėta 2026-05-09).
- [5] D. P. Kingma, J. Ba, „Adam: A method for stochastic optimization“, iš *3rd International Conference on Learning Representations (ICLR)*, 2015.
- [6] TensorFlow Authors. „Tensorflow: An end-to-end open source machine learning platform“. (2025), interaktyvus: <https://www.tensorflow.org/> (žiūrėta 2026-05-09).
- [7] OpenAI. „ChatGPT pokalbis: Sequential modelio Flatten sluoksnis perėjimui iš Conv2D į Dense“. Pokalbio nuoroda nepateikiama (privataus pokalbio archyvas). (2025), interaktyvus: <https://chat.openai.com/> (žiūrėta 2026-04-30).
- [8] OpenAI. „ChatGPT pokalbis: NumPy keturmačio tensoriaus formavimas iš vaizdų sąrašo“. Pokalbio nuoroda nepateikiama (privataus pokalbio archyvas). (2025), interaktyvus: <https://chat.openai.com/> (žiūrėta 2026-04-30).
- [9] OpenAI. „Chatgpt“. (2025), interaktyvus: <https://chat.openai.com/> (žiūrėta 2026-05-09).
- [10] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, „Dropout: A simple way to prevent neural networks from overfitting“, *Journal of Machine Learning Research*, tomas 15, puslapiai 1929–1958, 2014.
- [11] S. Ioffe, C. Szegedy, „Batch normalization: Accelerating deep network training by reducing internal covariate shift“, iš *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, puslapiai 448–456.

1 priedas.

Vaizdų klasifikavimo programa (main.py)

Žemiau pateikiamas pilnas vaizdų klasifikavimo programos kodas. Komentarai jame nurodo, kuriuose taškuose buvo naudoti dirbtinio intelekto įrankiai (žr. 5. skyrių).

% TODO: čia įklijuoti pilną 3Lab/main.py turinį

2 priedas. Laiko eilučių klasifikavimo programa (main_ts.py)

Žemiau pateikiamas pilnas laiko eilučių klasifikavimo programos kodas.

% TODO: čia įklijuoti pilną 3Lab/main_ts.py turinį